Industrial Robotic Arm

Srishti (230108055), Shrut Jain (230108053)
EE396 : Design Lab

April 24, 2026

Supervisor: Dr. Arun Aloshious

Department of Electronics and Electrical Engineering
Indian Institute of Technology, Guwahati
Guwahati, 781039

0 Contents

1 Introduction

Robotic arms are machines designed to perform tasks similar to a human arm. They can move, pick up objects, and place them with accuracy. These systems are widely used in industries to automate repetitive and precise work. This project was undertaken with the goal of designing and implementing a 4 Degree-of-Freedom (DoF) robotic arm capable of controlled position actuation via potentiometer-based input and encoder feedback.

The arm is designed in two stages. Stage 1 focuses on hardware assembly and closed-loop position control of individual joints using DC encoder motors driven by a PID controller running on an ESP32 microcontroller. Stage 2 extends this to motion mimicking, where positions recorded on a smaller reference structure are replicated on the arm in real time.

The project bridges theoretical concepts such as PID control, PWM motor driving, interrupt-driven encoder reading, and sensor calibration with practical embedded-systems engineering and mechanical design. Through this work, hands-on experience was gained in firmware development, motor characterisation, 3D-printed structural design, and perf-board circuit integration.

2 Problem Statement

2.1 Stage 1: Controlled Arm

Design and build a 4-DoF robotic arm controllable via:

- Physical input devices (potentiometers) mapped to each joint.

Each joint is driven by a DC encoder motor with closed-loop PID position control, ensuring accurate and repeatable angular positioning across the full range of motion.

2.2 Stage 2: Motion Mimicking

The arm was upgraded from Stage 1 to enable real-time motion replication using a reference structure fitted with potentiometers. The angular positions from the potentiometers are directly read by the ESP32 and mapped to the corresponding joints of the robotic arm, allowing it to mimic human-driven gestures in real time.

3 Algorithmic Flowchart

The following flowchart shows the overall control logic of the robotic arm system, including input reading, PID computation, and motor actuation.

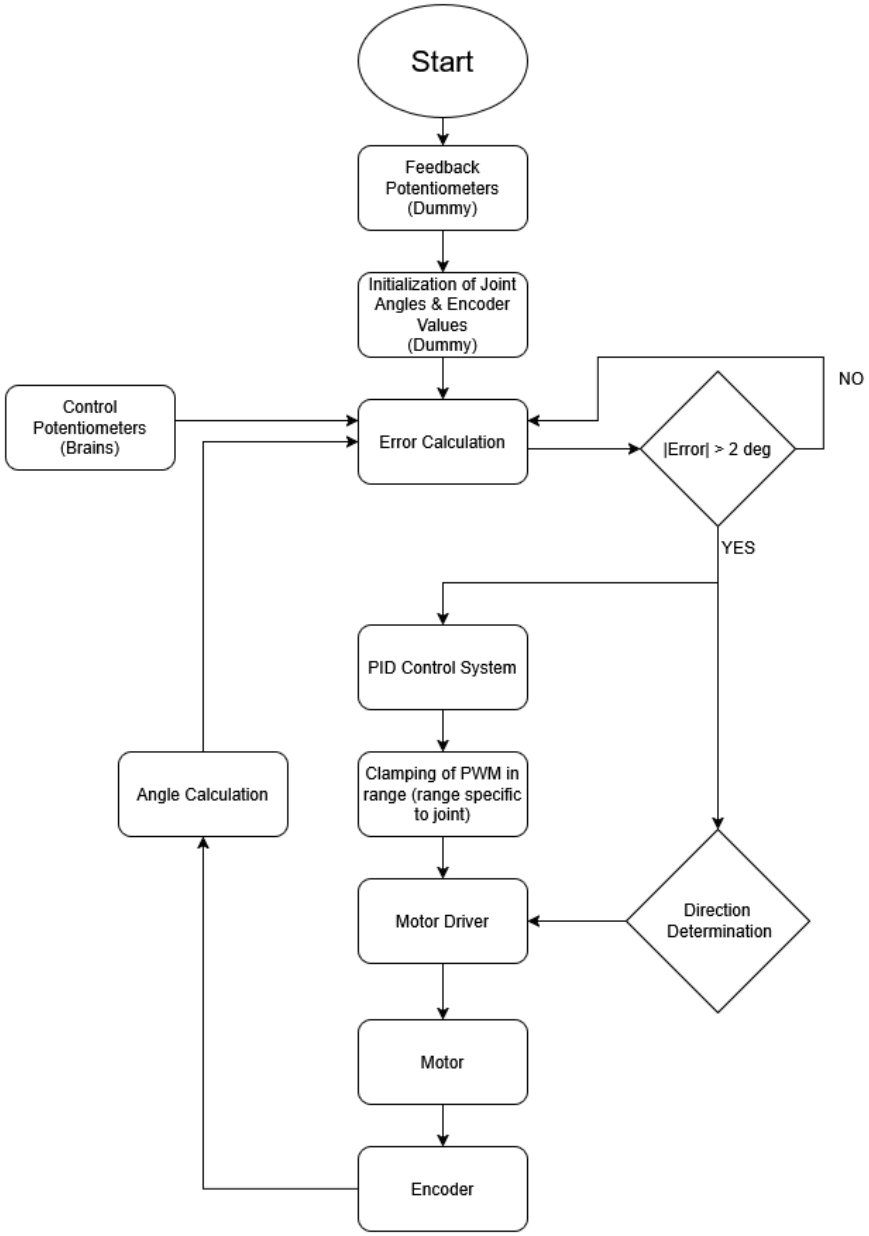

Figure 1: Algorithmic flowchart of the robotic arm control system.

4 Arm Architecture and Joint Naming

The arm consists of four rotational joints denoted S1 through S4, arranged as follows (from end-effector to base):

Joint	Description
S1	Wrist / End Effector
S2	Elbow
S3	Shoulder
S4	Base (rotation)

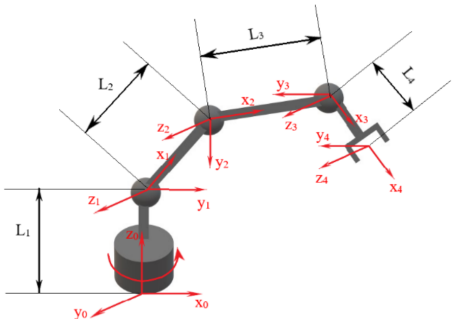

Figure 2: Joint diagram of the 4-DoF robotic arm.

5 Components Used

Table 1 lists all major components used in the project.

Table 1: List of Components

S.No.	Component	Description / Role
1	ESP32 Dev Module (×2)	Microcontroller; one per 2-motor subsystem
2	DC Encoder Motor (×4)	Actuators for joints S1–S4; PPR = 1340 (output shaft)
3	L293D Motor Driver IC (×2)	H-bridge driver; 600 mA per channel, 4.5–36 V motor supply
4	10 kΩ Potentiometer	Command input (target angle) and absolute feedback sensing
5	DC Power Supply (8 V)	Motor power; current-limited to 1.2 A per motor
6	Perf Board	Permanent circuit integration
7	SMA / Jumper Wires	Signal and power interconnects
8	3D-Printed PLA Structure	Arm links and joint housings

6 Design Theory

6.1 Motor and Encoder Characterisation

Each DC encoder motor has the following measured parameters:

- Pulses Per Revolution (encoder wheel): 12
- Pulses Per Revolution (output shaft): 1340
- Gear Ratio: $\frac{1340}{12} \approx 112$

The angular resolution per encoder pulse on the output shaft is:

$$\theta_{\text{pulse}} = \frac{360^\circ}{1340} \approx 0.269^\circ/\text{pulse} \tag{1}$$

6.2 PID Position Control

A discrete-time PID controller is implemented in firmware to drive each motor to a target angle θ_{ref} commanded by an input potentiometer. The control law is:

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de}{dt} \tag{2}$$

where $e(t) = \theta_{\text{ref}}(t) - \theta_{\text{actual}}(t)$ is the angular error, and K_p, K_i, K_d are the proportional, integral, and derivative gains respectively.

In discrete time with sampling interval Δt:

$$e_k = \theta_{\text{ref},k} - \theta_{\text{actual},k} \tag{3}$$
$$I_k = I_{k-1} + e_k\, \Delta t \tag{4}$$
$$D_k = \frac{e_k - e_{k-1}}{\Delta t} \tag{5}$$
$$u_k = K_p\, e_k + K_i\, I_k + K_d\, D_k \tag{6}$$

The signed output u_k determines motor direction and magnitude. The PWM duty cycle is constrained within a deadband to overcome static friction:

$$\text{PWM} = \text{clamp}(|u_k|,\ \text{PWM}_{\text{min}},\ 255) \tag{7}$$

Empirically, $\text{PWM}_{\text{min}} = 165$ was found to be the minimum duty cycle (out of 255 at 8-bit resolution) required to overcome static friction at 8 V supply, but depending on joints and arm's own inertia the values were incremented as mentioned below:
S1: 165
S2: 165
S3: 220
S4: 180

6.3 Potentiometer Calibration

The feedback potentiometer spans a physical range of 300° across its full electrical track. Due to manufacturing tolerances, the *true* maximum electrical angle may differ from 300°. A calibration procedure was developed:

1. Record encoder angle θ_{enc} and mapped pot angle θ_{pot} at rest.
2. Move the motor a known amount; record new values.
3. Compute the scale correction:

$$\theta_{\text{true max}} = 300 \times \frac{\Delta\theta_{\text{enc}}}{\Delta\theta_{\text{pot}}} \tag{8}$$

The corrected value replaces the 300 in the `map()` function, aligning the potentiometer's electrical scale with the encoder's ground truth. Based on this procedure and calculation $\theta_{\text{true max}}$ was 262.61 deg, which was satisfactory conclusion when implemented.

6.4 PWM and Motor Driver

The L293D H-bridge operates at 1.5 V (Input Low Voltage Max.) and 2.3 V (Input Low Voltage Min.) logic levels (compatible with ESP32) and drives motors from a 8 V supply. Decoupling capacitors across the motor terminals and power supply pins of the L293D reduce switching noise. PWM frequency is set to 20 kHz to minimise audible whine.

7 Pin Allocation and Circuit Design

Each ESP32 controls two motors. The pin allocation per ESP32 is:

Table 2: ESP32 Pin Allocation (per board, 2 motors)

GPIO	Function	Notes
4, 13	EN1, EN2 (PWM)	Speed control via L293D ENABLE
18, 19	IN1, IN2 (Motor 1)	Direction control
23, 25	IN1, IN2 (Motor 2)	Direction control
26, 27	Encoder A, B (Motor 1)	Interrupt-driven
32, 33	Encoder A, B (Motor 2)	Interrupt-driven
36, 39	Command Pot 1, 2	Input only (ADC), target angle
34, 35	Feedback Pot 1, 2	Input only (ADC), absolute position

GPIO pins 34, 35, 36, and 39 are input-only pins on the ESP32; they are used exclusively for the analog potentiometer readings. Pins 14 and 15 were avoided due to random PWM output at boot on some ESP32 variants.

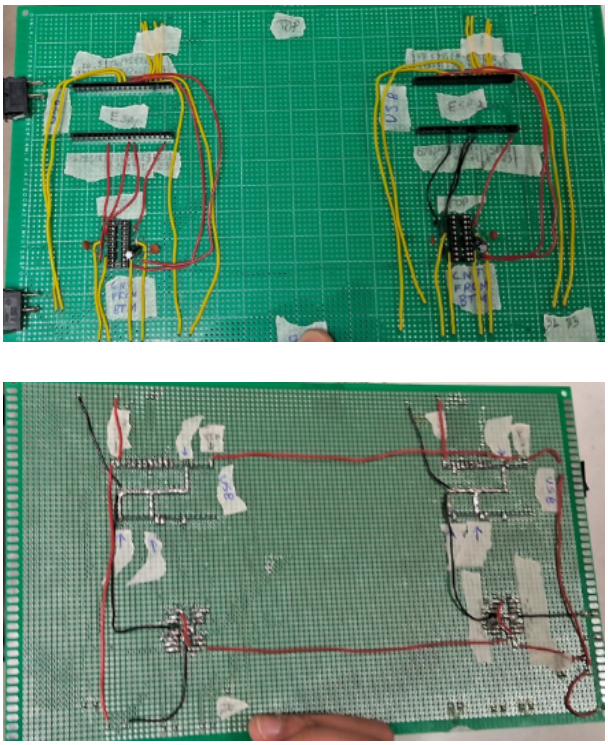

Figure 3: Perfboard wiring layout showing motor drivers, microcontroller connections, and power distribution.

8 Firmware Architecture

The firmware is written in C++ using the Arduino framework on the ESP32. It is structured around three functional layers: encoder ISRs, the PID control loop, and the serial communication handler.

8.1 Encoder Interrupt Service Routines

Encoder pulses are counted using hardware interrupts on the rising edge of Channel A, with the direction determined by reading Channel B:

```
void IRAM_ATTR readEncoder1() {
    if (digitalRead(encB1)) encoderCount1++;
    else                    encoderCount1--;
}
```

Listing 1: Encoder ISR for Motor 1

The `IRAM_ATTR` attribute ensures the ISR runs from IRAM, avoiding cache miss delays. Interrupts are briefly disabled when reading the volatile counter to prevent race conditions.

8.2 Absolute Position Initialisation

On power-up, the encoder counter is seeded from the feedback potentiometer to avoid an arbitrary reference:

```
float initialAngle1 = map(analogRead(feedbackPotPin1), 0, 4095, 0,
   262.609);
encoderCount1 = initialAngle1 / degPerPulse;
zeroCount1    = 0;
```

Listing 2: Absolute position initialisation on startup

This means that even if the arm is moved while unpowered, the system recovers the true joint angle at the next power-on.

8.3 Main Control Loop

The main loop runs at approximately 100 Hz (10 ms delay). Each iteration:

1. Reads command potentiometers and maps ADC values to degrees.
2. Reads current angle from encoder counters.
3. Computes PID output.
4. Constrains PWM and applies motor direction.
5. Transmits telemetry over Serial.

```
float error1 = target1 - actual1;

integral1   += error1 * dt;
derivative1  = (error1 - previousError1) / dt;
output1      = Kp*error1 + Ki*integral1 + Kd*derivative1;
previousError1 = error1;
```

```
7
8  int pwm1 = constrain(abs((int)output1), 165, 255);
9
10 if      (error1 >  2)  motor1Forward(pwm1);
11 else if (error1 < -2)  motor1Backward(pwm1);
12 else                   motor1Stop();
```

Listing 3: PID computation and motor actuation (Motor 1)

A deadband of $\pm 2^\circ$ prevents motor chatter near the setpoint.

8.4 Tuned PID Parameters

Parameter(S1,S2)	Value
K_p	2.67
K_i	0.02
K_d	0.15

Parameter(S3,S4)	Value
K_p	2.0
K_i	0.02
K_d	0.10

9 3D Mechanical Design

The structural links of the arm were designed in CAD and fabricated using PLA 3D-printing. Key design decisions include:

- **Base (S4):** Houses the base rotation motor. A cutout on the side panel routes cables from S4 upward through the structure.
- **Shoulder link (S3 to S4):** Rectangular link with cutouts for cable routing and a slot for the S4 motor shaft.
- **Upper-arm link (S2 to S3):** Connects shoulder to elbow; accommodates S3 shaft and provides potentiometer mounting pockets.
- **Forearm / End-Effector (S1, S2):** Wrist joint at S1; elbow at S2. Designed similarly to the upper-arm link.

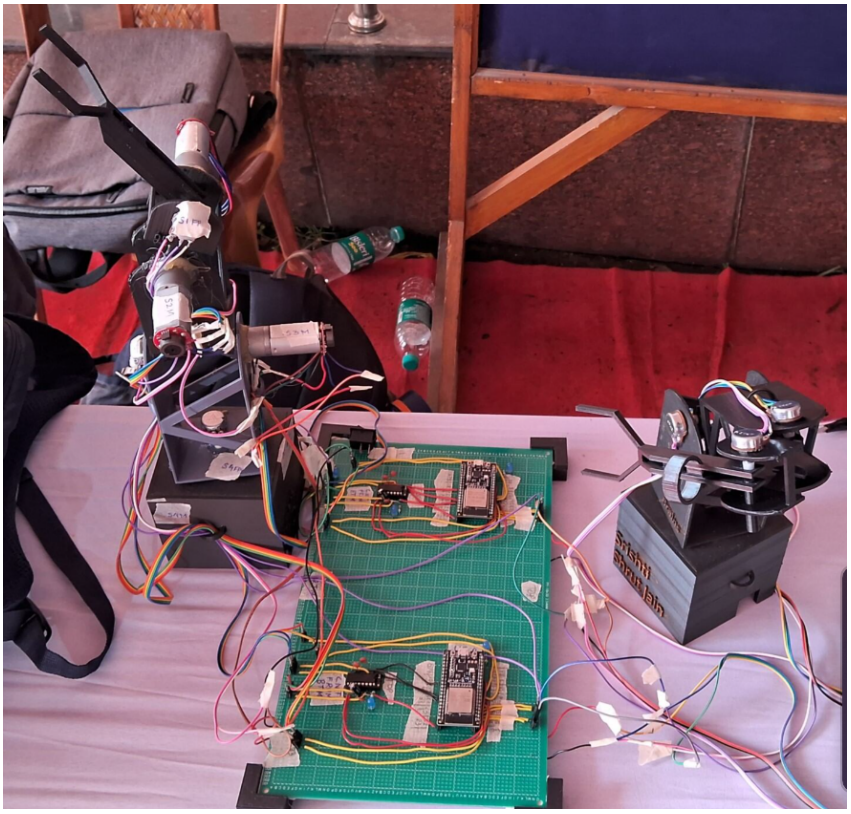

Figure 4: 2nd Stage Design

10 Results and Observations

- **Position tracking:** Both motors under individual PID control converged to the commanded angle within $\pm 2^\circ$ of the setpoint with no steady-state oscillation under no-load conditions.
- **Absolute initialisation:** Seeding the encoder from the feedback potentiometer on power-up eliminated the arbitrary-start-position problem. The arm correctly assumes its current physical pose after a power cycle.
- **Minimum PWM:** A threshold of PWM = 165/255 was confirmed experimentally for reliable motor start-up at an 8 V supply.
- **Gear ratio:** Encoder PPR of 1340 at the output shaft was verified, corresponding to a gear ratio of approximately 112, consistent with theoretical expectations.
- **Potentiometer accuracy:** The 10 kΩ potentiometers combined with a 12-bit ADC provided sufficient angular resolution. Noise levels were below one ADC LSB after simple integer averaging, eliminating the need for additional filtering.
- **Mechanical performance:** The 3D-printed joints provided adequate stiffness for the arm's weight. Cable routing through link cutouts ensured smooth motion and prevented wire entanglement during operation.
- **Live mimicking response:** The slave arm successfully replicated the motion of the master arm in near real time. Motion tracking was smooth under moderate speeds with no abrupt discontinuities.

- **Tracking consistency:** Joint angle mapping between the master and slave arms remained consistent across multiple trials, maintaining alignment with only minor deviations.
- **System limitations:** At higher speeds, slight tracking errors and lag increased due to motor response limits and communication latency, indicating scope for further optimisation.

11 Challenges Faced

- **GPIO boot behaviour:** Several ESP32 GPIOs (e.g., 14, 15) produce transient PWM signals at boot, causing unintended motor movement. This was resolved by selecting only stable GPIO pins that do not exhibit such behaviour.
- **Potentiometer range mismatch:** The assumed 300° range in the firmware did not match the actual electrical limits of the potentiometers. A calibration procedure was implemented to accurately map the potentiometer readings to true joint angles.
- **Mechanical tolerances (3D printing):** Variations in 3D printing dimensions led to fitting issues and slight misalignments in joints. Proper tolerance considerations were required in the CAD design to ensure smooth assembly and motion.
- **Mechanical backlash:** Backlash (especially slip) was observed at motor-to-link couplings due to loose fits and shaft tolerances. This was reduced using trial and error by using shaft connectors of various tolerances and choosing the one which fits the best.

12 Discussion

The robotic arm achieved reliable closed-loop position control using encoder feedback and PID control, with steady-state error within 2°. The control pipeline—from potentiometer input to PWM motor actuation—operated smoothly and provided stable joint motion.

Compared to open-loop alternatives, the system offers better accuracy and repeatability due to continuous error correction. The two-ESP32 architecture also enabled efficient control of multiple joints, demonstrating a scalable design for multi-DoF robotic systems.

13 Conclusion

A 4-DoF robotic arm with PID-based closed-loop position control was successfully designed, built, and tested. Key achievements include:

- Encoder-based angular measurement with interrupt-driven counting.
- Absolute position recovery on power-up via feedback potentiometers.
- Stable PID position control with $\pm 2°$ accuracy.
- 3D-printed structural design with motor integration and cable management.
- Stage 2 implementation with live mimicking

The project demonstrated that effective closed-loop robotic control can be achieved using low-cost, off-the-shelf components, and that a systematic approach to hardware characterisation, firmware design, and mechanical integration is essential for reliable performance.

14 Future Work

- Complete Stage 2 FSM implementation position sequence record and replay.
- Add an OLED display for live joint angle readout.
- Design a weatherproof, perf-board-free PCB for robust long-term use.
- Integrate a 5th DoF for pick-and-place end effector operations.
- Explore capacitive touch buttons as an alternative to mechanical switches.

15 Project Repository

The complete source code, circuit schematics, and documentation for this project are available on GitHub:

https://github.com/shrutjain18/Robotic-Arm-Design-Lab

The repository includes:

- Firmware code for the microcontroller
- Circuit diagrams and wiring references
- 3D design files
- Contraption Images